hashlock.

Security Audit

Finceptor (DeFi)

May 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

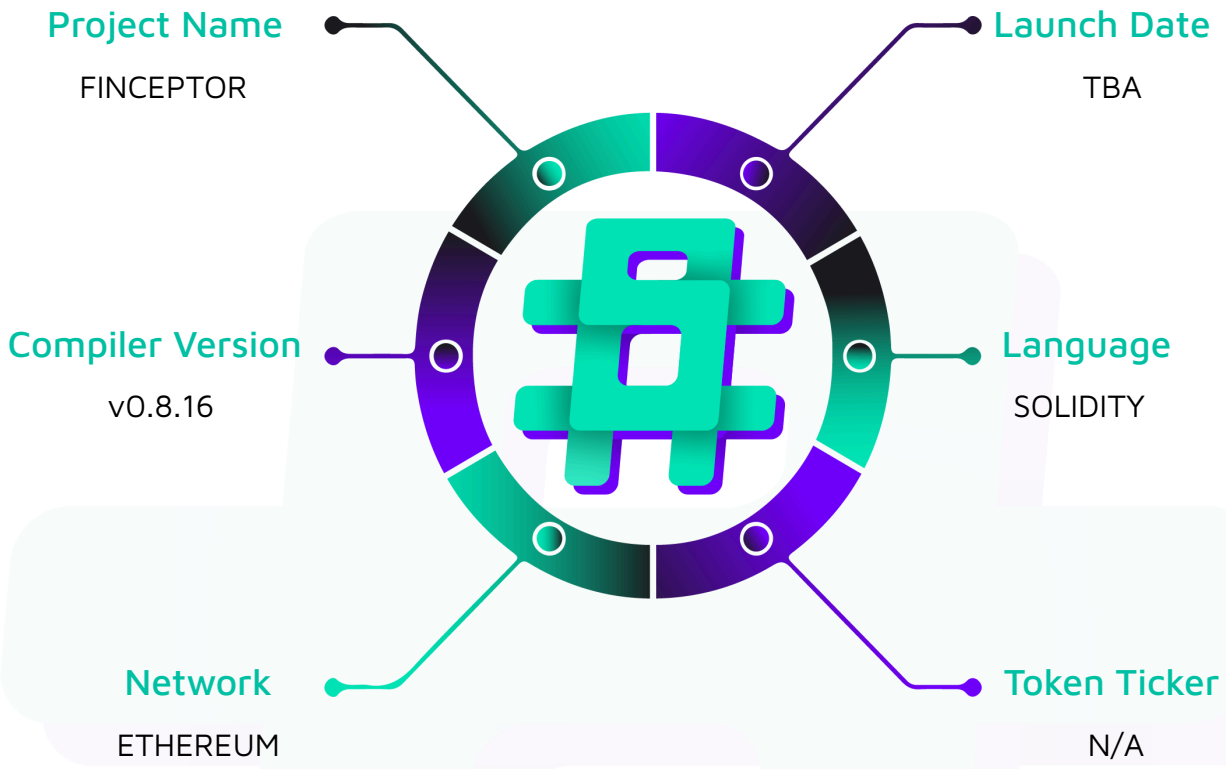
The Finceptor team partnered with Hashlock to conduct a security audit of their AbstractReleaser.sol, AbstractVesting.sol, Releaser.sol, Vesting.sol, VestingClaimTge.sol and VestingPartialClaim.sol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

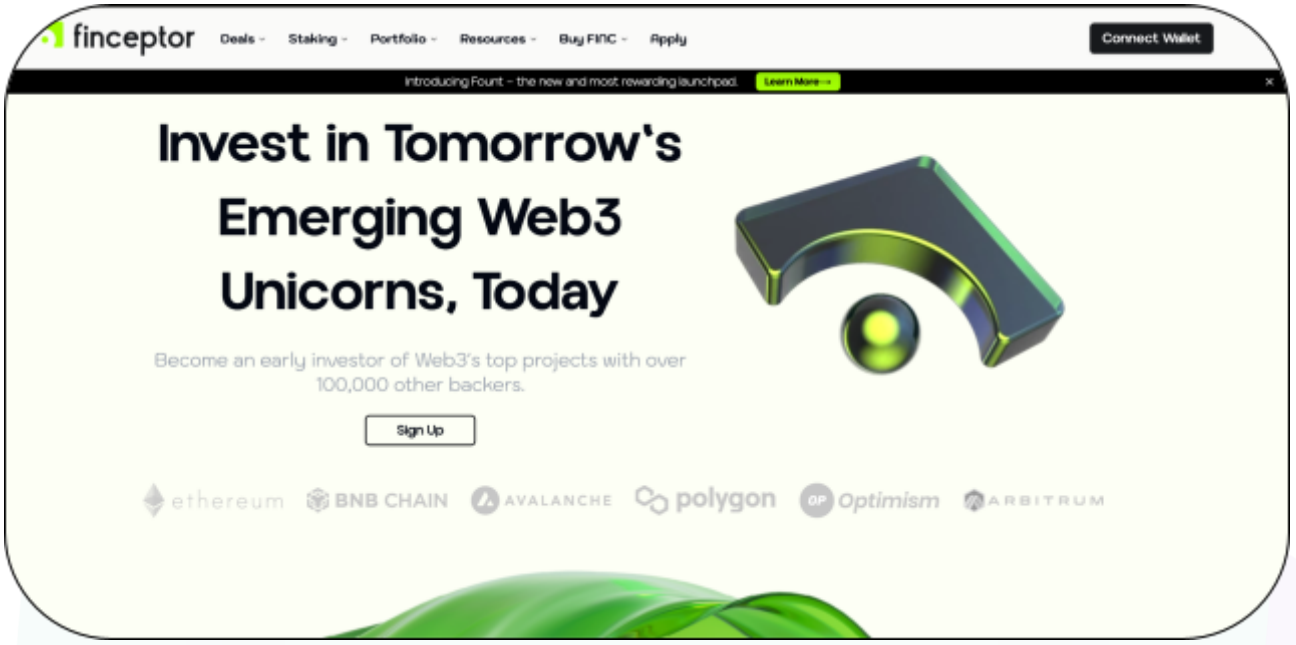
Finceptor is a decentralized finance (DeFi) platform designed to democratize access to early-stage Web3 investments, enabling retail investors to participate in high-potential projects with minimal capital, starting as low as $1. It offers a suite of innovative financial tools, including Liquidity Vaults for pre-launch token liquidity, Initial Token Offerings (ITOs) for token launches, and Follow-on Token Offerings (FTOs) for post-launch capital growth, all aimed at addressing liquidity challenges in DeFi while fostering protocol-owned liquidity. Finceptor curates top-tier Web3 projects and integrates features like capital protection policies, refund options, and a dual-token economic model to streamline investments and enhance user security. The platform also incentivizes community engagement through social quests and staking rewards.

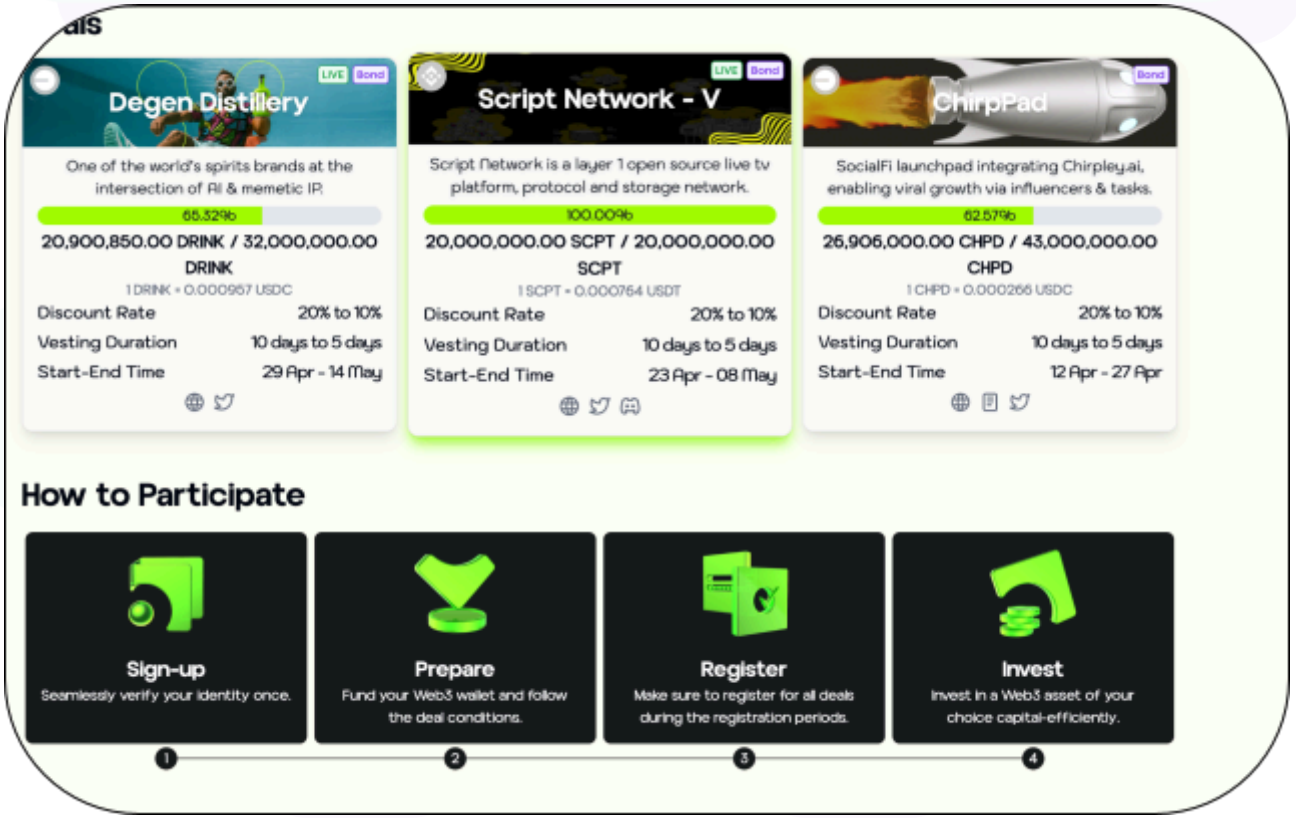
Project Name: Finceptor
Project Type: Defi
Compiler Version: 0.8.16
Website: https://finceptor.app/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Finceptor, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Finceptor Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**May, 2025**
Contract 1	AbstractVesting.sol
Contract 2	AbstractReleaser.sol
Contract 3	Releaser.sol
Contract 4	Vesting.sol
Contract 5	VestingPartialClaim.sol
Contract 6	VestingClaimTge.sol
Audited GitHub Commit Hash	60ececcb605ca118f502fba53c491a5ac4c34c39
Fix Review GitHub Commit Hash	bb7d18772a1b246f78dbd61c2c826fe14177fab0

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

5 Low severity vulnerabilities

2 Gas Optimisations

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AbstractReleaser.sol Manages ERC20 token vesting by allowing a designated beneficiary to claim vested tokens over time, tracking released amounts via the `_erc20Released` state variable. It calculates releasable tokens using the abstract `vestedAmount` function (to be implemented in derived contracts), permits owners to emergency-withdraw tokens via `emergencyWithdraw`, and allows beneficiaries to retroactively reduce the recorded released amount with `reduceReleasedAmount`.	**Contract achieves this functionality.**
AbstractVesting.sol Manages ERC20 token vesting with role-based access, enabling admins to allocate shares, set claim and airdrop fees, and configure refund periods. It integrates a `Releaser` contract for vesting schedules, and supports refunds during a configurable window for unclaimed shares. Admins can also batch manage shares and airdrops, trigger emergency withdrawals, and automate airdrops via Chainlink Keepers. The contract enforces pausing, reentrancy guards, and tracks released/refunded amounts. Fees are sent to designated reserves, and ETH overpayments are refunded to users.	**Contract achieves this functionality.**
Releaser.sol Implements a linear ERC20 token vesting schedule with periodic unlocks, allowing a beneficiary to claim tokens	**Contract achieves this functionality.**

proportionally over time based on elapsed periods relative to total duration. Owners can adjust vesting parameters (start time, duration, period length), while the `vestedAmount` function calculates unlocked tokens via a linear formula. It also tracks released amounts, and inherits emergency withdrawals/release functions from `AbstractReleaser`. Vesting progresses incrementally per configured periods, ensuring tokens become claimable in chunks aligned with the periodicity until the total allocation is unlocked.	
Vesting.sol Token vesting system with refund functionality, deploying a linear `Releaser` contract to manage vesting schedules (cliff, duration, periodic unlocks). The contract will initialize vesting parameters such as TGE, refund period, etc. and can deploy or update the `Releaser`. Beneficiaries claim vested tokens post TGE or request refunds during the refund period, receiving refundToken proportional to their unclaimed shares. Refunds transfer unclaimed tokens (from both Vesting and Releaser balances) to a refund reserve and issue equivalent refundToken to users.	**Contract achieves this functionality.**
VestingClaimTge.sol Manages token vesting with refund capabilities tied to a token generation event (TGE). Built on top of `AbstractVesting`, it initializes with key vesting parameters like cliff, duration, and refund periods. Participants receive token shares that vest over time, and they may request refunds within a defined period post-TGE, provided they haven't claimed or refunded yet. The contract also allows the admin to deploy custom	**Contract achieves this functionality.**

Releaser contracts to manage claim schedules. Refunds are handled by removing a user's shares, transferring vested and releasable tokens to a refund reserve, and returning pegged tokens from the refund reserve to the user, while emitting relevant events for transparency.	
VestingPartialClaim.sol Enables partial token claiming, allowing participants to claim a customizable percentage of their unlocked tokens after a token generation event (TGE), while also supporting time-limited refunds. Built on top of AbstractVesting, the contract manages user shares and released amounts, and requires a fee for each partial claim, which is forwarded to a designated fee reserve. Participants can specify any percentage (in basis points) of their available releasable tokens to claim, with strict bounds enforced. Refunds are permitted during a defined refund window post TGE, where a user can be reimbursed for their unclaimed portion of both TGE and locked allocations, with shares adjusted accordingly. The admin can also deploy new Releaser contracts to customize release schedules, and the contract emits relevant events to ensure operational transparency.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Finceptor, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Finceptor smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] Releaser.sol - Missing startTimestamp validation

Description

`Releaser.sol`'s constructor and `Releaser.sol::updateStartTime()` function fail to validate that `startTimestamp` is in the future, allowing vesting to begin or end immediately if a past timestamp is provided.

Vulnerability Details

Without a check like `startTimestamp >= block.timestamp`, an incorrect or maliciously set start time (e.g., a historical timestamp) triggers vesting calculations from that point. This bypasses the intended vesting schedule, as periods are counted from the outdated start time, enabling beneficiaries to claim tokens prematurely. The issue is compounded by the updateStartTime function, which lets the owner retroactively modify the start time even after deployment.

Impact

Accidental or malicious use of a past start time leads to unintended token releases, violating the vesting schedule's purpose. Funds reserved for future distribution become immediately claimable.

Recommendation

Add a validation in the constructor:

```
require(
    startTimestamp >= block.timestamp,
    "Releaser: start must be in the future"
);
```

Additionally, restrict `updateStartTime()` to prevent retroactive adjustments by requiring `startTimestamp > block.timestamp` if the vesting hasn't begun.

Note

Finceptor's team informed Hashlock that this finding is not considered a threat under their current trust model. The function in question is restricted to the contract owner, so it cannot be exploited by malicious users. Additionally, the ability to set a `startTime` in the past is intentional and necessary to support scenarios where projects may be hacked and need to redeploy their tokens. In such cases, unrestricted configuration of the vesting schedule is required.

Status

Acknowledged

[M-02] AbstractVesting,Vesting, VestingClaimTge, VestingPartialClaim - Refund token allowance not ensured

Description

The `_refundUser` function attempts to transfer `refundToken` tokens from the `refundReserve` to the user via `refundToken.safeTransferFrom(refundReserve, _account, shareToRefund)`. However, this operation requires prior approval from the `refundReserve` to allow the vesting contracts to spend its tokens. The contract does not verify or enforce this allowance, risking transaction failures during refund execution.

Vulnerability Details

The `safeTransferFrom` function relies on the `refundReserve` having granted an allowance to the vesting contracts equal to or exceeding `shareToRefund`. If the `refundReserve` has not explicitly approved the vesting contracts to spend its tokens, the transfer will revert. The current implementation assumes this allowance is preconfigured off chain, introducing a dependency on external conditions that are neither validated nor enforced within the contract, creating a silent point of failure.

Impact

Refund transactions will irreversibly fail when the required allowance is missing, preventing users from claiming refunds and leaving their funds indefinitely locked in the contract. This undermines the core refund functionality, erodes user trust, and could lead to financial losses or disputes, particularly during critical periods like the refund window.

Recommendation

Implement an allowance check within the `_refundUser` function or during contract initialization to ensure the `refundReserve` has granted sufficient approval.

Note

Finceptor's team informed Hashlock that this finding does not require any changes to the contract logic. If the refund reserve (their multisig wallet) hasn't approved the necessary amount, the refund transaction will fail regardless of any checks. This approval step must occur after the contract is deployed, making it impossible to enforce at the time of contract creation. Additionally, adding validation to the refund function would not mitigate the issue. The Finceptor team confirmed that this process will be handled manually.

Status

Acknowledged

Low

[L-01] AbstractReleaser.sol - Unrestricted renounceOwnership() call

Description

The contract inherits from OpenZeppelin's `Ownable` contract, which includes the `renounceOwnership()` function. This function allows the current owner to irreversibly relinquish ownership, setting the owner address to `address(0)`.

Impact

In the current implementation, `renounceOwnership()` is not overridden or restricted, meaning any accidental or intentional call by the owner will permanently remove owner control over the contract.

Recommendation

If ownership should never be renounced (common for fixed-governance or permanently-administered contracts), override and disable It by adding the following function:

```
function renounceOwnership() public override onlyOwner {

    revert("Renouncing ownership is disabled.");

}
```

Status

Acknowledged

[L-02] AbstractReleaser.sol - Use Ownable2Step contract rather than Ownable contract

Description

`Ownable` contract from OpenZeppelin is used in the `contracts/vesting/AbstractReleaser.sol` contract. The primary concern with the `Ownable` contract's `transferOwnership` function is its immediacy and lack of verification. Once called, ownership is instantly transferred to the specified address without any confirmation from the recipient.

Impact

If the new owner's address is entered incorrectly, the contract's control could be lost or handed over to an unintended entity, leading to potential security risks or loss of functionality.

Recommendation

To mitigate this risk, it's advisable to use OpenZeppelin's `Ownable2Step` contract, this contract enhances the ownership transfer process by implementing a two-step mechanism.

This two-step process ensures that ownership is only transferred if the new owner explicitly accepts it, thereby preventing accidental transfers to incorrect addresses.

Status

Acknowledged

[L-03] Contracts - Lack of emitting an event

Description

The below functions are lacking event emission:

`AbstractReleaser.sol::emergencyWithdraw()`

`AbstractReleaser.sol::reduceReleasedAmount()`

`AbstractVesting.sol::setAirdropStatus()`

`AbstractVesting.sol::updateIterationNumber()`

`AbstractVesting.sol::setRefundPeriod()`

`AbstractVesting.sol::updateReleaser()`

`AbstractVesting.sol::setRefundReserve()`

`AbstractVesting.sol::setFeeReserve()`

`AbstractVesting.sol::setClaimFee()`

`AbstractVesting.sol::setAirdropFee()`

`AbstractVesting.sol::updateTimes()`

`AbstractVesting.sol::addAirdrop()`

`AbstractVesting.sol::removeAirdrop()`

`AbstractVesting.sol::requestAirdrop()`

`Releaser.sol::updateStartTime()`

`Releaser.sol::updateDuration()`

`Releaser.sol::updatePeriods()`

`Vesting.sol::deployReleaser()`

`VestingClaimTge.sol::deployReleaser()`

`VestingPartialClaim.sol::deployReleaser()`

Impact

The absence of events for these critical functions may result in a lack of transparency, making it difficult for users to track important changes in the contract. Emitting events can ensure that users are informed.

Recommendation

Add relevant events in the functions.

Status

Acknowledged

[L-04] AbstractVesting.sol::setClaimFee, AbstractVesting.sol::setAirdropFee - Uncapped claim and airdrop fees

Description

The `AbstractVesting.sol` contract allows admin to set arbitrary claim and airdrop fees via `setClaimFee` and `setAirdropFee` functions without enforcing maximum limits. This lack of constraints means fees can be set to exorbitant values, potentially rendering the claiming or airdrop process economically unfeasible for users.

Impact

Uncapped fees enable admins to disrupt core functionalities, leading to denial-of-service (DoS) for users, loss of funds, or protocol abandonment. Malicious admin could exploit this to extort users, while accidental misconfigurations could irreversibly break the system, eroding trust and violating the contract's purpose.

Recommendation

Implement fee caps (e.g., `MAX_FEE` constants) enforced in `setClaimFee` and `setAirdropFee`, ensuring fees cannot exceed predefined limits. Consider decentralizing fee adjustments via governance votes or multi-sig approvals. Additionally, emit events for fee changes to enhance transparency and allow users to monitor adjustments.

Status

Resolved

[L-05] AbstractVesting.sol - Division by zero revert when _totalShares = 0

Description

The contract contains multiple functions (`_shareOf()`, `_pendingPayment()`, and `_removeShares()`) that perform division operations using `_totalShares` as the denominator. If `_totalShares` is equal to zero or is reduced to zero (e.g., via admin functions like `removeShares`), these divisions will revert due to division-by-zero errors, halting core functionalities like checking claimable tokens etc.

Impact

The default revert message for divide-by-zero is non-descriptive. Without custom checks, developers and users struggle to understand failures

Recommendation

Handle the case when `_totalShares = 0` in functions (`AbstractVesting.sol::shareOf()`, `AbstractVesting.sol::_pendingPayment()`, and `AbstractVesting.sol::_removeShares()`) and prevent division by zero revert.

Status

Acknowledged

Gas

[G-01] AbstractVesting.sol - Unnecessary initialisation of default value

Description

In Solidity, uninitialized variables automatically assume their default values (e.g., `0`, `false`, `0x0`, depending on the data type). Explicitly setting them to these defaults is considered an anti-pattern and results in unnecessary gas consumption.

`AbstractVesting.sol Line 53:`

```
bool internal iterationActive = false;
```

Recommendation

Refactor the code as follow:

```
--  bool internal iterationActive = false;

++  bool internal iterationActive;
```

Status

Resolved

[G-02] AbstractVesting.sol - Unnecessary conditional check should be removed

Description

A conditional check to verify whether a `uint` input is greater than or equal to zero is unnecessary, as Solidity guarantees that the `uint` data type is always zero or positive by definition.

`AbstractVesting.sol Line 439 & Line 458:`

```
require(_start >= 0, "Vesting: start is negative");
```

Recommendation

Remove the conditional checks to save deployment gas.

Status

Resolved

QA

[Q-01] AbstractVesting.sol - Typos

Description

`remaingCount` on function `AbstractVesting.sol::autoAirdrop()` is misspelled, it should be `remainingCount`.

`addressCountPerIteartion` variable in contract `AbstractVesting.sol` is misspelled, it should be `addressCountPerIteration`.

Recommendation

Fix the typos.

Status

Resolved

[Q-02] VestingClaimTge.sol - Wrong Event Logging

Description

The `Refunded` event in function `VestingClaimTge.sol::_refundUser` logs `notClaimed` (vesting tokens), but the actual refund is in `peggedToSend` (refund tokens). This could cause confusion.

Recommendation

Emit `peggedToSend` instead of `notClaimed` in `Refunded` event.

Status

Acknowledged

[Q-03] VestingPartialClaim.sol - Wrong Event Logging

Description

The `Refunded` event in function `VestingPartialClaim.sol::_refundUser` logs `refundTGETokens + refundLockedTokens`, but the actual refund is in `shareToRefund` (refund tokens). This could cause confusion.

Recommendation

Emit `shareToRefund` instead of `refundTGETokens + refundLockedTokens` in `Refunded` event.

Status

Acknowledged

[Q-04] Vesting.sol - Wrong Event Logging

Description

The `Refunded` event in function `Vesting.sol::_refundUser` logs `notClaimed`, but the actual refund is in `share` (refund tokens). This could cause confusion.

Recommendation

Emit `share` instead of `notClaimed` in `Refunded` event.

Status

Acknowledged

Centralisation

Finceptor values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Finceptor project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au